

27. Remove Element

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` in-place. The order of the elements may be changed. Then return the number of elements in `nums` which are not equal to `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.

Return `k`.

Code:

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int k=0;
        for(int i=0;i<nums.length;i++){
            if(nums[i]!=val){
                nums[k]=nums[i];
                k++;
            }
        }
        return k;
    }
}
```

[344. Reverse String](#)

Write a function that reverses a string. The input string is given as an array of characters `s`.

You must do this by modifying the input array [in-place](#) with $O(1)$ extra memory.

Code:

```
class Solution {
    public void reverseString(char[] s) {
        int l=0;
        int r=s.length-1;
        while(l<r){
            char temp=s[l];
            s[l]=s[r];
            s[r]=temp;
            l++;
            r--;
        }
    }
}
```

```
}  
}
```

[88. Merge Sorted Array](#)

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be *stored inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`.

Code:

```
class Solution {  
    public void merge(int[] nums1, int m, int[] nums2, int n) {  
        int i = m - 1;  
        int j = n - 1;  
        int k = m + n - 1;  
  
        while (i >= 0 && j >= 0) {  
            if (nums1[i] > nums2[j]) {  
                nums1[k--] = nums1[i--];  
            } else {  
                nums1[k--] = nums2[j--];  
            }  
        }  
  
        while (j >= 0) {  
            nums1[k--] = nums2[j--];  
        }  
    }  
}
```

[75. Sort Colors](#)

You are given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Code:

```
class Solution {
    public void sortColors(int[] nums) {
        int l = 0;
        int r = nums.length-1;
        int i = 0;
        while(i<=r){
            if(nums[i] == 0){
                int temp = nums[i];
                nums[i] = nums[l];
                nums[l] = temp;
                l++;
                i++;
            }else if(nums[i] == 2){
                int temp = nums[i];
                nums[i] = nums[r];
                nums[r] = temp;
                r--;
            }else{
                i++;
            }
        }
    }
}
```

[287. Find the Duplicate Number](#)

Given an array of integers nums containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only **one repeated number** in nums, return *this repeated number*.

You must solve the problem **without** modifying the array nums and using only constant extra space.

Code:

```
class Solution {
    public int findDuplicate(int[] nums) {
        int n=nums.length;
        boolean[] visited = new boolean[n+1];

        for(int i=0; i<n; i++) {
            if(visited[nums[i]]) {
                return nums[i];
            } else {
                visited[nums[i]]=true;
            }
        }
    }
}
```

```

    }
    return 0;
}

```

189. Rotate Array

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Code:

```

class Solution {
public void rotate(int[] nums, int k) {
    k=k%nums.length;
    reverse(nums,0,nums.length-1);
    reverse(nums,0,k-1);
    reverse(nums,k,nums.length-1);
}
static void reverse(int[] nums,int l,int r){
    while (l < r) {
        int temp = nums[l];
        nums[l++] = nums[r];
        nums[r--] = temp;
    }
}
}

```